

Private content with shared deal state

VINSS runs message encryption in the browser and records authorized business actions on Canton. The wallet connects the user to their ledger Party.

Layer	Responsibility	Code entry point
Browser app	Invite and QR flow, chat, offer, delivery and escrow actions.	frontend/app/; frontend/lib/
OpenMLS WASM	KeyPackages, group admission, message encryption and decryption.	wasm/vinss_mls/; frontend/lib/openmls/
Canton messaging	Signed peer binding and transport of encrypted envelopes.	daml/Vinss/Messaging.daml; src/messaging/canton/
Deal & Escrow	Agreement, funding reference, fulfillment, review and settlement authority.	daml/Vinss/Deal.daml
Token registry	Allocation creation and authorized token transfer.	src/canton/token-wallet.ts

Invitation to an encrypted room

The creator shares an invite URL or its QR. The joiner connects a wallet and publishes the signed registration and KeyPackage. The creator resolves the Party/installation binding and admits the peer through MLS Welcome/Commit. A URL starts admission; wallet binding supplies the peer identity. The normal join flow does not ask the user to type peer IDs.

Private chat and groups

The sender encrypts content in the browser. Canton carries EncryptedMessage ciphertext and envelope metadata to designated Parties. Recipients decrypt locally. Private rooms support deal actions. Group rooms support encrypted conversation and membership display, with creator-assisted admission.

Runtime condition

The creator must remain available in the original browser for admission. The UI supports up to 32 group members; the recorded automated group scenario uses three members.

Where text, keys and payment records live

Message plaintext stays in browser storage after decryption. The ledger carries message ciphertext and the business metadata required by the contracts.

Data	Storage and visibility
Sent and received message text	Plaintext in local IndexedDB, scoped by network, Party, installation and room.
MLS keys and checkpoints	Local IndexedDB, encrypted at rest using a non-exportable device key.
Message ciphertext and envelope	Canton messaging contracts, visible to designated Parties and authorized infrastructure.
Detailed offer terms	Inside the encrypted conversation. The proposal records a terms hash.
Amount, asset, Parties and deal state	Canton deal contracts according to signatory and observer roles.

What each privacy layer does

OpenMLS protects message content before it reaches the transport. Canton controls which Parties can observe contract data and exercise choices. Authorized ledger infrastructure can still see ciphertext and metadata. Deal amounts and Parties are not hidden from their authorized ledger viewers.

Local history and recovery

Reloading the same browser can restore stored plaintext history and encrypted MLS checkpoints. Clearing site storage removes both. Another device does not inherit them. History and checkpoint writes are not atomic, and simultaneous tabs do not share an MLS write lock. These are current recovery and reliability limits.

Verification scope

The local integration tests use real OpenMLS WASM over simulated ledger/wallet access. They check two-person and group messages, reload, replay, delayed reads and ciphertext-only submissions. A current live browser-wallet run remains a separate verification step.

Deal & Escrow (Rekber)

The payer creates a Token Standard Allocation. VINSS validates its link to the deal, carries the reference through review and settles after approval.

Contract stage	Authorized action	Result
DealProposal	Acceptor accepts the offer.	DealAgreement
DealAgreement	Reviewer/payer funds with an existing allocation.	DealEscrow
DealEscrow	Fulfiller/payee submits delivery hash.	DealFulfillment
DealFulfillment	Reviewer approves or requests revision.	FulfillmentApproval or DealRevisionRequest
DealRevisionRequest	Fulfiller submits revised delivery.	DealFulfillment, next round
FulfillmentApproval	Fulfiller executes Settle.	Token transfer and SettlementReceipt

Checks at funding and settlement

FundEscrow matches the allocation admin, asset, amount, deal reference, sender, receiver and executor to the agreement. Settle exercises Allocation_ExecuteTransfer using registry-supplied context and records receiver holding references. VINSS references the allocation rather than taking custody of the funds.

Acceptance, funding and payment are distinct states

The UI performs acceptance and allocation/funding as separate wallet steps. An accepted agreement can remain unfunded if the second step fails. Approval creates authority for the VINSS settlement path. Completion requires the transfer and receipt, not only a chat status message.

Implemented boundary

No dispute, arbitration, refund or group escrow choice exists in this module. Other actions on the allocation remain governed by the token implementation. The recorded live evidence is a 1 CC DevNet settlement on 6 October 2026.

Sources and implementation

Architecture and runtime limits
 Messaging verification

Complete escrow guide
 Live settlement identifiers